Ai chosen: Claude,geimini
//platform Gemini website engine
//Yunjingma Assigment04
//author yunjinig ma
//colloboration:Chatgpt halps explaination about the meaning of somepart of >
// small amoubt of the code are gemini code orginal.

Prompt

Write a complete C++ program for EECS 348 Software Engineering I Assignment 4.

Requirements:

1. Use object-oriented programming with classes and objects, rather than implementing the solution entirely with standalone functions.
2. Solve Sudoku puzzles using recursive depth-first search with backtracking.
3. Each Sudoku puzzle is a 9×9 grid. Every row, column, and 3×3 block must contain the digits 1 through 9 exactly once.
4. Read five input files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. Each file contains a Sudoku puzzle, with underscores (_) representing empty cells.
5. For each puzzle, print the input filename, the original puzzle, and every valid solution. If no solution exists, print "No solution found".
6. If a puzzle has multiple solutions, print all of them.
7. Include appropriate error handling for file-reading errors and invalid puzzle input.
8. Write the entire implementation in a single source file named Assignment4.cpp.
9. Include detailed comments explaining the code, the major blocks, and the purpose of each important statement.
10. Use standard C++ features compatible with the EECS Cycle server environment.

Provide the complete source code and briefly explain the class design, recursive backtracking algorithm, time complexity, space complexity, and any assumptions about the input file format.

CLAUDE
Platform claude website engine

```
/*
 * Assignment4.cpp
 * EECS 348 - Software Engineering I - Assignment 4
 *
 * Sudoku solver using object-oriented design and recursive depth-first
 * search (DFS) with backtracking.
 *
```

```
 * Program flow:
 *   1. For each of puzzle1.txt ... puzzle5.txt:
 *   2. Read the puzzle from the file into a SudokuBoard (validating it).
 *   3. Print the filename and the original puzzle.
 *   4. Use SudokuSolver to find ALL solutions via backtracking.
 *   5. Print every solution, or "No solution found".
 *
 * Classes:
 *   PuzzleError   - exception type for file / format / rule violations.
 *   SudokuBoard   - stores a 9x9 grid; reads, validates, and prints it.
 *   SudokuSolver  - recursive DFS + backtracking; collects all solutions.
 *   PuzzleRunner  - ties reading, solving, and printing together per file.
 *
 * Compile (C++11 or newer):  g++ -std=c++11 -Wall -Wextra -o Assignment4 Assignment4.cpp
 * Run:                       ./Assignment4
 */

#include <cctype>     // std::isspace
#include <cstddef>    // std::size_t
#include <fstream>    // std::ifstream for reading puzzle files
#include <iostream>   // std::cout, std::cerr
#include <sstream>    // std::ostringstream for building filenames/messages
#include <stdexcept>  // std::runtime_error base class
#include <string>     // std::string
#include <vector>     // std::vector to hold solutions

// ---------------------------------------------------------------------------
// PuzzleError
// A dedicated exception type so the program can distinguish puzzle-related
// problems (missing file, bad characters, wrong size, rule violations)
// from any other failure. It carries a human-readable message.
// ---------------------------------------------------------------------------
class PuzzleError : public std::runtime_error {
public:
    explicit PuzzleError(const std::string& message)
        : std::runtime_error(message) {}
};

// ---------------------------------------------------------------------------
// SudokuBoard
// Represents one 9x9 Sudoku grid. A cell holds 0 when empty (shown as '_'
// in input/output) or a digit 1..9 when filled.
// Responsibilities: loading from file, validating the givens, checking
// whether a placement is legal, and printing.
```

```
// ----------------------------------------------------------------------
class SudokuBoard {
public:
    static const int SIZE = 9;       // Grid is SIZE x SIZE
    static const int BOX_SIZE = 3;   // Each block is BOX_SIZE x BOX_SIZE
    static const int EMPTY = 0;      // Internal marker for an empty cell

    // Constructor: start with a completely empty board.
    SudokuBoard() {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c)
                cells_[r][c] = EMPTY;
    }

    // Read a puzzle from the named file.
    // Format: 81 meaningful characters, each a digit '1'-'9' or '_' (empty).
    // Whitespace/newlines are ignored, so the usual 9-lines-of-9 layout works,
    // as does a layout with spaces between cells.
    // Throws PuzzleError if the file cannot be opened/read, contains an
    // illegal character, has the wrong number of cells, or the givens
    // already violate Sudoku rules.
    void loadFromFile(const std::string& filename) {
        std::ifstream in(filename.c_str());   // Open file for reading
        if (!in.is_open()) {                  // Open failed: missing/unreadable
            throw PuzzleError("Cannot open file '" + filename + "'.");
        }

        int count = 0;       // How many cell characters we have consumed
        int lineNumber = 1;  // Current line, used for helpful error messages
        char ch;             // Current character

        // Read one character at a time so we can accept any whitespace layout.
        while (in.get(ch)) {
            if (ch == '\n') {                 // Track line numbers
                ++lineNumber;
                continue;
            }
            if (std::isspace(static_cast<unsigned char>(ch))) {
                continue;                     // Skip spaces, tabs, '\r', etc.
            }

            // Anything beyond 81 cells means the file is too large.
            if (count >= SIZE * SIZE) {
                throw PuzzleError("File '" + filename +
```

```
                    "' contains more than 81 cells.");
        }

        int row = count / SIZE;           // Row index of this cell
        int col = count % SIZE;           // Column index of this cell

        if (ch == '_') {
            cells_[row][col] = EMPTY;     // Underscore = empty cell
        } else if (ch >= '1' && ch <= '9') {
            cells_[row][col] = ch - '0';  // Convert char digit to int
        } else {
            // Illegal character (e.g. '0', letters, punctuation).
            std::ostringstream msg;
            msg << "Invalid character '" << ch << "' on line "
                << lineNumber << " of '" << filename
                << "' (expected 1-9 or '_').";
            throw PuzzleError(msg.str());
        }
        ++count;
    }

    // A bad stream state other than normal end-of-file means a read error.
    if (in.bad()) {
        throw PuzzleError("I/O error while reading '" + filename + "'.");
    }

    // Too few cells (includes empty files).
    if (count != SIZE * SIZE) {
        std::ostringstream msg;
        msg << "File '" << filename << "' has " << count
            << " cells; expected 81.";
        throw PuzzleError(msg.str());
    }

    // Finally make sure the starting digits do not already conflict.
    validateGivens();
}

// Accessor: value at (row, col).
int get(int row, int col) const { return cells_[row][col]; }

// Mutator: set value at (row, col). Used by the solver; 0 clears the cell.
void set(int row, int col, int value) { cells_[row][col] = value; }
```

```
    // Is it legal to place 'value' at (row, col) given the current board?
    // Checks the row, the column, and the 3x3 block for the same digit.
    // (The target cell itself is ignored so this also works for validating
    // digits that are already placed.)
    bool isSafe(int row, int col, int value) const {
        // Check the row and column in one pass.
        for (int i = 0; i < SIZE; ++i) {
            if (i != col && cells_[row][i] == value) return false;  // row clash
            if (i != row && cells_[i][col] == value) return false;  // col clash
        }
        // Find the top-left corner of the 3x3 block containing (row, col).
        int boxRow = (row / BOX_SIZE) * BOX_SIZE;
        int boxCol = (col / BOX_SIZE) * BOX_SIZE;
        // Check all 9 cells of that block.
        for (int r = boxRow; r < boxRow + BOX_SIZE; ++r) {
            for (int c = boxCol; c < boxCol + BOX_SIZE; ++c) {
                if ((r != row || c != col) && cells_[r][c] == value)
                    return false;                           // block clash
            }
        }
        return true;  // No conflicts found
    }

    // Print the board to the given stream, 9 rows of 9 characters,
    // using '_' for empty cells (the same format as the input files).
    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (cells_[r][c] == EMPTY) out << '_';
                else                       out << cells_[r][c];
                if (c < SIZE - 1) out << ' ';   // Space between cells
            }
            out << '\n';
        }
    }

private:
    int cells_[SIZE][SIZE];   // The grid; 0 = empty, 1..9 = filled

    // Verify every pre-filled digit is consistent with the Sudoku rules.
    // If two equal digits share a row, column, or block, the puzzle is
    // invalid input and we report which cell is at fault.
    void validateGivens() const {
        for (int r = 0; r < SIZE; ++r) {
```

```
            for (int c = 0; c < SIZE; ++c) {
                int v = cells_[r][c];
                if (v != EMPTY && !isSafe(r, c, v)) {
                    std::ostringstream msg;
                    msg << "Invalid puzzle: digit " << v << " at row "
                        << (r + 1) << ", column " << (c + 1)
                        << " conflicts with another given digit.";
                    throw PuzzleError(msg.str());
                }
            }
        }
    }
};

// ---------------------------------------------------------------------------
// SudokuSolver
// Finds ALL solutions of a SudokuBoard using recursive depth-first search
// with backtracking:
//   - Pick the next empty cell (scanning row by row).
//   - Try digits 1..9; for each legal digit, place it and recurse.
//   - When the recursive call returns (success or dead end), UNDO the
//     placement (backtrack) and try the next digit.
//   - If no empty cell remains, the board is a complete valid solution;
//     save a copy and keep searching for others.
// A safety cap on the number of stored solutions prevents runaway output
// for nearly-empty puzzles (an empty grid has ~6.7e21 solutions).
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    // board        : the puzzle to solve (copied, so the original is untouched)
    // maxSolutions : stop after this many solutions are found
    explicit SudokuSolver(const SudokuBoard& board,
                          std::size_t maxSolutions = 1000)
        : working_(board), maxSolutions_(maxSolutions), limitReached_(false) {}

    // Run the search. Results are available via solutions() afterwards.
    void solve() {
        solutions_.clear();
        limitReached_ = false;
        search();  // Begin recursion at the first empty cell
    }

    // All solutions discovered.
    const std::vector<SudokuBoard>& solutions() const { return solutions_; }
```

```
    // True if the search stopped early because the cap was hit.
    bool limitReached() const { return limitReached_; }

private:
    SudokuBoard working_;                 // Board mutated during the search
    std::vector<SudokuBoard> solutions_;  // Completed boards found so far
    std::size_t maxSolutions_;            // Cap on stored solutions
    bool limitReached_;                   // Set when the cap stops the search

    // Locate the first empty cell in row-major order.
    // Returns true and fills row/col if one exists; false if the board is full.
    bool findEmptyCell(int& row, int& col) const {
        for (int r = 0; r < SudokuBoard::SIZE; ++r) {
            for (int c = 0; c < SudokuBoard::SIZE; ++c) {
                if (working_.get(r, c) == SudokuBoard::EMPTY) {
                    row = r;
                    col = c;
                    return true;
                }
            }
        }
        return false;  // No empty cells left
    }

    // The recursive DFS / backtracking routine.
    void search() {
        int row = 0, col = 0;

        // BASE CASE: no empty cell remains, so every cell was filled legally.
        if (!findEmptyCell(row, col)) {
            solutions_.push_back(working_);          // Save a copy of the solution
            if (solutions_.size() >= maxSolutions_)  // Respect the cap
                limitReached_ = true;
            return;                                  // Backtrack to look for more
        }

        // RECURSIVE CASE: try each candidate digit in this empty cell.
        for (int digit = 1; digit <= 9; ++digit) {
            if (working_.isSafe(row, col, digit)) {  // Prune illegal choices early
                working_.set(row, col, digit);       // Choose: place the digit
                search();                            // Explore: recurse deeper
                working_.set(row, col, SudokuBoard::EMPTY);  // Un-choose: backtrack
                if (limitReached_) return;           // Stop everything if capped
```

```
            }
        }
        // If no digit worked, we simply return: the caller will backtrack.
    }
};

// ---------------------------------------------------------------------------
// PuzzleRunner
// Handles the end-to-end processing of one puzzle file: read, print,
// solve, and print results. Errors are caught and reported here so one
// bad file does not stop the remaining puzzles from being processed.
// ---------------------------------------------------------------------------
class PuzzleRunner {
public:
    // Process a single puzzle file and write output to 'out'.
    void run(const std::string& filename, std::ostream& out) const {
        out << "============================================\n";
        out << "File: " << filename << "\n";
        out << "============================================\n";

        try {
            SudokuBoard puzzle;
            puzzle.loadFromFile(filename);   // May throw PuzzleError

            out << "Original puzzle:\n";
            puzzle.print(out);
            out << '\n';

            SudokuSolver solver(puzzle);     // Build solver with a copy of the puzzle
            solver.solve();                  // Run DFS + backtracking

            const std::vector<SudokuBoard>& sols = solver.solutions();

            if (sols.empty()) {
                out << "No solution found\n";
            } else {
                out << "Number of solutions found: " << sols.size();
                if (solver.limitReached())
                    out << " (search stopped at the solution limit)";
                out << "\n\n";
                // Print every solution found.
                for (std::size_t i = 0; i < sols.size(); ++i) {
                    out << "Solution " << (i + 1) << ":\n";
                    sols[i].print(out);
```

```
                out << '\n';
            }
        }
    } catch (const PuzzleError& e) {
        // Expected problems: bad file, bad format, invalid puzzle.
        out << "Error: " << e.what() << "\n\n";
    } catch (const std::exception& e) {
        // Any unexpected standard exception (e.g. std::bad_alloc).
        out << "Unexpected error: " << e.what() << "\n\n";
    }
  }
};

// ---------------------------------------------------------------------------
// main
// Processes puzzle1.txt through puzzle5.txt in order.
// ---------------------------------------------------------------------------
int main() {
  const int NUM_PUZZLES = 5;   // Number of input files to process
  PuzzleRunner runner;         // Object that handles each file

  for (int i = 1; i <= NUM_PUZZLES; ++i) {
    std::ostringstream name;            // Build "puzzleN.txt"
    name << "puzzle" << i << ".txt";
    runner.run(name.str(), std::cout);   // Read, solve, and print
  }
  return 0;
}
```

GEMINI: platform: gemini online website engine version

```
/**
 * @file Assignment4.cpp
 * @brief Sudoku Solver using Object-Oriented Programming and Recursive Backtracking.
 * @course EECS 348 Software Engineering I - Assignment 4
 *
 * Description:
 * This program reads five Sudoku puzzle files (puzzle1.txt through puzzle5.txt),
 * validates the input data, and solves each puzzle using a recursive depth-first
 * search algorithm with backtracking. It prints the input filename, the original
 * puzzle, and all valid solutions found (or a message if no solution exists).
 */

#include <iostream>
```

```
#include <fstream>
#include <string>
#include <vector>

/**
 * @class SudokuSolver
 * @brief Encapsulates a 9x9 Sudoku board, file loading logic, validation checks,
 *        and the recursive backtracking solving algorithm.
 */
class SudokuSolver {
private:
    int board[9][9];       // 9x9 grid representing the Sudoku puzzle (0 denotes empty cells)
    std::string filename;  // The input file name associated with this puzzle instance

    /**
     * @brief Checks if placing a number at a specific cell violates Sudoku rules.
     * @param row Current row index (0-8)
     * @param col Current column index (0-8)
     * @param num The digit to check (1-9)
     * @return true if the placement is valid, false otherwise.
     */
    bool isValid(int row, int col, int num) const {
        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (board[row][i] == num) return false;
            if (board[i][col] == num) return false;
        }

        // Check 3x3 subgrid constraints
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (board[startRow + r][startCol + c] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    /**
     * @brief Recursive helper function implementing DFS with backtracking to find all solutions.
```

```
     * @param row Current row index
     * @param col Current column index
     * @param solutionCount Reference tracker counting valid solutions found
     */
    void solveRecursive(int row, int col, int &solutionCount) {
        // Base case: If row reaches 9, we have successfully filled the entire grid
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            printBoard();
            return;
        }

        // Determine coordinates for the next cell
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already pre-filled, skip to the next cell
        if (board[row][col] != 0) {
            solveRecursive(nextRow, nextCol, solutionCount);
            return;
        }

        // Try placing digits 1 through 9 in the empty cell
        for (int num = 1; num <= 9; ++num) {
            if (isValid(row, col, num)) {
                board[row][col] = num; // Make choice

                // Recurse to the next cell
                solveRecursive(nextRow, nextCol, solutionCount);

                board[row][col] = 0;   // Backtrack (undo choice)
            }
        }
    }

public:
    /**
     * @brief Constructor that initializes the Sudoku solver with a target filename.
     * @param fname The name of the input text file.
     */
    SudokuSolver(const std::string &fname) : filename(fname) {
        // Initialize board with zeros
        for (int i = 0; i < 9; ++i) {
```

```
            for (int j = 0; j < 9; ++j) {
                board[i][j] = 0;
            }
        }
    }

    /**
     * @brief Loads and parses the Sudoku puzzle from the input file.
     * @return true if successfully loaded and validated, false on error.
     */
    bool loadPuzzle() {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << "\n";
            return false;
        }

        int row = 0;
        std::string line;

        while (getline(file, line)) {
            // Ignore empty lines
            if (line.empty()) continue;

            if (row >= 9) {
                std::cerr << "Error: File " << filename << " contains more than 9 rows.\n";
                return false;
            }

            int col = 0;
            for (char ch : line) {
                // Skip whitespace characters
                if (ch == ' ' || ch == '\t' || ch == '\r' || ch == '\n') continue;

                if (col >= 9) {
                    std::cerr << "Error: Row " << row + 1 << " in " << filename << " exceeds 9
columns.\n";
                    return false;
                }

                // Parse cell value: '_' or '0' or '.' represents empty cell
                if (ch == '_' || ch == '0' || ch == '.') {
                    board[row][col] = 0;
                } else if (ch >= '1' && ch <= '9') {
```

```
                board[row][col] = ch - '0';
            } else {
                std::cerr << "Error: Invalid character '" << ch << "' in " << filename << ".\n";
                return false;
            }
            col++;
        }

        if (col != 9) {
            std::cerr << "Error: Row " << row + 1 << " in " << filename << " has " << col << "
columns (expected 9).\n";
            return false;
        }
        row++;
    }

    file.close();

    if (row != 9) {
        std::cerr << "Error: File " << filename << " contains only " << row << " rows (expected
9).\n";
        return false;
    }

    return true;
}

/**
 * @brief Prints the current state of the Sudoku board in a formatted grid.
 */
void printBoard() const {
    for (int r = 0; r < 9; ++r) {
        if (r > 0 && r % 3 == 0) {
            std::cout << "------+-------+------\n";
        }
        for (int c = 0; c < 9; ++c) {
            if (c > 0 && c % 3 == 0) {
                std::cout << "| ";
            }
            if (board[r][c] == 0) {
                std::cout << "_ ";
            } else {
                std::cout << board[r][c] << " ";
            }
```

```
            }
            std::cout << "\n";
        }
        std::cout << "\n";
    }

    /**
     * @brief Executes the solver on the loaded puzzle and prints results.
     */
    void solve() {
        std::cout << "========================================\n";
        std::cout << "Input Filename: " << filename << "\n";
        std::cout << "Original Puzzle:\n";
        printBoard();

        int solutionCount = 0;
        solveRecursive(0, 0, solutionCount);

        if (solutionCount == 0) {
            std::cout << "No solution found.\n";
        } else {
            std::cout << "Total solutions found: " << solutionCount << "\n";
        }
        std::cout << "========================================\n\n";
    }
};

/**
 * @brief Main execution function that processes puzzle1.txt through puzzle5.txt.
 */
int main() {
    // Array containing the names of the five required puzzle files
    std::string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Iterate through each puzzle file and solve it
    for (int i = 0; i < 5; ++i) {
        SudokuSolver solver(puzzleFiles[i]);
```

```
        // Load puzzle and check for reading/format errors before solving
        if (solver.loadPuzzle()) {
            solver.solve();
        } else {
            std::cout << "Skipping " << puzzleFiles[i] << " due to input errors.\n\n";
        }
    }

    return 0;
}
```

Gemini

Correctness:

Gemini version solves the problem; it can clearly identify how many solutions there are or if there is no solution.

The logic is to check row by row, then check the 3*3 grid by 3*3

The isvalid function checks if any numbers are in the position; it will raise an error

But it doesn't check whether the number given already has a conflict or not

The execution time

The worst time should be (9^E) it depends on how many blanks are on the board

By running 5 Sudokus 100 times, we got the Gemini version is 0.19 seconds;

usr/bin/time -f "Total: %e seconds" sh -c 'for i in $(seq 1 100); do ./Gemini_revised > /dev/null; done'

Total: 0.19 seconds

The space complexity

Gemini modifies the Sudoku board directly and prints the solution when it finds it, so it doesn't need to store every solution in memory. The main space is used by the recursive of checking, solving the problem, reading the file, and filling the answer on the Sudoku board

For maintainability

Gemini is pretty easy to maintain because most functions are contained in one class, and the class functions are clearly separated. And Gemini is easier to read compared with the Claude version . There are already a few comments generated. But it didn't comment on the main functions, isValid and solutionCheck, so we have to add more comments on it

Claude:

Correctness: The Claude one can also run the function correctly, using separate classes for the Sudoku board solver, puzzle runner, and input errors

It also checks for number conflicts

And the Claude version defaults to saving 1000 solutions for each Sudoku board

Execution time:

The time should be O(9^E) the time depends on how many blanks are left on the initial board

By the reality

y602m419@engr1005c-16:~/EECS348/Assignment_Ma_04$ /usr/bin/time -f "Total: %e seconds" sh -c 'for i in $(seq 1 100); do ./Claudeversion > /dev/null; done'
Total: 0.20 seconds

A little longer than Gemini

The space complexity

Claude stores the solution in as std:vetor<sudokuboard> so it stores solution which need more memory compared the the gemini
And if there's a lot of blank and muti different solution the storage and the space complexity are way more larger than Gemini

4maintainability

The claude program separates responsibility in muti different classes
The board class nanage the puzzle and validation
Sloverperforms class are for the search of the soulution and handling the error of the sudoku as repetition of the number the number of row or cloumn not meet the requirement
But still it didn't comment on the main function of the code

choice :
As the better time and space complexity i will choose gemini
The correctness
The major of the code are correct
But it didn't check the original sudoku board if already have the error on it
I would add a validateInitialBoard() function to check all given numbers before solving This would allow the program to earlier get the no solution sudoku board

2 execution time
The algorithm itself can't be faster a lot the only way to improve is the check the original sudoku board it cut the time which waste by checking the board that already have the errors

3 space complexity
The algorithm tiself can't be improve a lot
only way to improve is the check the original sudoku board it cut the time which waste by checking the board that already have the errors

4 the minatainability
For the maintainability
So i will make it more clearly

And add more comment for the main function because it’s not very clear to let the reader understand the code